

An Apple Library Primer

Developer Tools & Services

General

Linker

This thread has been locked by a moderator; it no longer accepts new replies.



DTS Engineer

Apple

Created

Sep '22

Replies

0

Boosts

0

Views

15k

Participants

1

Apple's library technology has a long and glorious history, dating all the way back to the origins of Unix. This does, however, mean that it can be a bit confusing to newcomers. This is my attempt to clarify some terminology.

If you have any questions or comments about this, start a new thread and tag it with *Linker* so that I see it.

Share and Enjoy

Quinn "The Eskimo!" @ Developer Technical Support @ Apple

```
let myEmail = "eskimo" + "1" + "@" + "apple.com"
```

An Apple Library Primer

Apple's tools support two related concepts:

- **Platform** — This is the platform itself; macOS, iOS, iOS Simulator, and Mac Catalyst are all platforms.
- **Architecture** — This is a specific CPU architecture used by a platform. `arm64` and `x86_64` are both architectures.

A given architecture might be used by multiple platforms. The most obvious example of this `arm64`, which is used by all of the platforms listed above.

Code built for one platform will not work on another platform, even if both platforms use the same architecture.

Code is usually packaged in either a Mach-O file or a static library. **Mach-O** is used for executables (`MH_EXECUTE`), dynamic libraries (`MH_DYLIB`), bundles (`MH_BUNDLE`), and object files (`MH_OBJECT`). These can have a variety of different extensions; the only constant is that `.o` is always used for a Mach-O containing an object file. Use `otool` and `nm` to examine a Mach-O file. Use `vtool` to quickly determine the platform for which it was built. Use `size` to get a summary of its size. Use `dyld_info` to get more details about a dynamic library.

IMPORTANT All the tools mentioned here are documented in man pages. For information on how to access that documentation, see [Reading UNIX Manual Pages](#). There's also a Mach-O [man page](#), with basic information about the file format.

Many of these tools have old and new variants, using the `-classic` suffix or `llvm-` prefix, respectively. For example, there's `nm-classic` and `llvm-nm`. If you run the original name for the tool, you'll get either the old or new variant depending on the version of the currently selected tools. To explicitly request the old or new variants, use `xcrun`.

The term **Mach-O image** refers to a Mach-O that can be loaded and executed without further processing. That includes executables, dynamic libraries, and bundles, but not object files.

A **dynamic library** has the extension `.dylib`. You may also see this called a shared library.

A **framework** is a bundle structure with the `.framework` extension that has both compile-time and run-time roles:

- At compile time, the framework combines the library's headers and its stub library (stub libraries are explained below).
- At run time, the framework combines the library's code, as a Mach-O dynamic library, and its associated resources.

The exact structure of a framework varies by platform. For the details, see [Placing Content in a Bundle](#).

macOS supports both frameworks and standalone dynamic libraries. Other Apple platforms support frameworks but not standalone dynamic libraries.

Historically these two roles were combined, that is, the framework included the headers, the dynamic library, and its resources. These days Apple ships different frameworks for each role. That is, the macOS SDK includes the compile-time framework and macOS itself includes the run-time one. Most third-party frameworks continue to combine these roles.

A **static library** is an archive of one or more object files. It has the extension `.a`. Use `ar`, `libtool`, and `ranlib` to inspect and manipulate these archives.

The **static linker**, or just the **linker**, runs at build time. It combines various inputs into a single output. Typically these inputs are object files, static libraries, dynamic libraries, and various configuration items. The output is most commonly a Mach-O image, although it's also possible to output an object file. The linker may also output metadata, such as a link map (see [Using a Link Map to Track Down a Symbol's Origin](#)).

The linker has seen three major implementations:

- `ld` — This dates from the dawn of Mac OS X.
- `ld64` — This was a rewrite started in the 2005 timeframe. Eventually it replaced `ld` completely. If you type `ld`, you get `ld64`.
- `ld_prime` — This was introduced with Xcode 15. This isn't a separate tool. Rather, `ld` now supports the `-ld_classic` and `-ld_new` options to select a specific implementation.

Note During the Xcode 15 beta cycle these options were `-ld64` and `-ld_prime`. I continue to use those names because the definition of *new* changes over time (some of us still think of `ld64` as the new linker ;-).

The **dynamic linker** loads Mach-O images at runtime. Its path is `/usr/lib/dyld`, so it's often referred to as `dyld`, `dyld`, or `DYLD`. Personally I pronounced that *dee-lid*, but some folks say *di-lid* and others say *dee-why-el-dee*.

IMPORTANT Third-party executables must use the standard dynamic linker.

Other Unix-y platforms support the notion of a statically linked executable, one that makes system calls directly. This is not supported on Apple platforms. Apple platforms provide binary compatibility via system dynamic libraries and frameworks, not at the system call level.

Note Apple platforms have vestigial support for custom dynamic linkers (your executable tells the system which dynamic linker to use via the `LC_LOAD_DYLINKER` load command). This facility originated on macOS's ancestor platform and has never been a supported option on any Apple platform.

The dynamic linker has seen 4 major revisions. See WWDC 2017 Session 413 (referenced below) for a discussion of versions 1 through 3. Version 4 is basically a merging of versions 2 and 3.

The `dyld` [man page](#) is chock-full of useful info, including a discussion of how it finds images at runtime.

Every dynamic library has an **install name**, which is how the dynamic linker identifies

the library. Historically that was the path where you installed the library. That's still true for most system libraries, but nowadays a third-party library should use an rpath-relative install name. For more about this, see [Dynamic Library Identification](#).

Mach-O images are **position independent**, that is, they can be loaded at any location within the process's address space. Historically, Mach-O supported the concept of **position-dependent** images, ones that could only be loaded at a specific address. While it may still be possible to create such an image, it's no longer a good life choice.

Mach-O images have a default **load address**, also known as the *base address*. For modern position-independent images this is 0 for library images and 4 GiB for executables (leaving the bottom 32 bits of the process's address space unmapped). When the dynamic linker loads an image, it chooses an address for the image and then **rebases** the image to that address. If you take that address and subtract the image's load address, you get a value known as the **slide**.

Xcode 15 introduced the concept of a **mergeable library**. This is a dynamic library with extra metadata that allows the linker to embed it into the output Mach-O image, much like a static library. Mergeable libraries have many benefits. For all the backstory, see WWDC 2023 Session 10268 [Meet mergeable libraries](#). For instructions on how to set this up, see [Configuring your project to use mergeable libraries](#).

If you put a mergeable library into a framework structure you get a **mergeable framework**.

Xcode 15 also introduced the concept of a **static framework**. This is a framework structure where the framework's dynamic library is replaced by a static library.

Note It's not clear to me whether this offers any benefit over creating a mergeable framework.

Earlier versions of Xcode did not have proper static framework support. That didn't stop folks trying to use them, which caused all sorts of weird build problems.

A **universal binary** is a file that contains multiple architectures for the same platform. Universal binaries always use the **universal binary format**. Use the `file` command to learn what architectures are within a universal binary. Use the `lipo` command to manipulate universal binaries.

A universal binary's architectures are either all in Mach-O format or all in the static library archive format. The latter is called a **universal static library**.

A universal binary has the same extension as its non-universal equivalent. That means a `.a` file might be a static library or a universal static library.

Most tools work on a single architecture within a universal binary. They default to the architecture of the current machine. To override this, pass the architecture in using a command-line option, typically `-arch` or `--arch`.

An **XCFramework** is a single document package that includes libraries for any combination of platforms and architectures. It has the extension `.xcframework`. An XCFramework holds either a framework, a dynamic library, or a static library. All the elements must be the same type. Use `xcodebuild` to create an XCFramework. For specific instructions, see [Xcode Help > Distribute binary frameworks > Create an XCFramework](#).

Historically there was no need to code sign libraries in SDKs. If you shipped an SDK to another developer, they were responsible for re-signing all the code as part of their distribution process. Xcode 15 changes this. You should sign your SDK so that a developer using it can verify this dependency. For more details, see WWDC 2023 Session 10061 [Verify app dependencies with digital signatures](#) and [Verifying the origin of your XCFrameworks](#).

A **stub library** is a compact description of the contents of a dynamic library. It has the extension `.tbd`, which stands for *text-based description* (TBD). Apple's SDKs include stub libraries to minimise their size; for the backstory, read [this post](#). Use the `tapi` tool to create and manipulate stub libraries. In this context *TAPI* stands for a *text-based API*, an alternative name for TBD. Oh, and on the subject of `tapi`, I'd be remiss if I didn't mention `tapi-analyze`!

Stub libraries currently use YAML format, a fact that's relevant when you try to [interpret linker errors](#). If you're curious about the format, read the `tapi-tbdv4` [man page](#). There's also a JSON variant documented in the `tapi-tbdv5` [man page](#).

Note Back in the day stub libraries used to be Mach-O files with all the code removed (`MH_DYLIB_STUB`). This format has long been deprecated in favour of TBD.

Historically, the system maintained a **dynamic linker shared cache**, built at runtime from its working set of dynamic libraries. In macOS 11 and later this cache is included in the OS itself. Libraries in the cache are no longer present in their original locations on disk:

```
1 % ls -lh /usr/lib/libSystem.B.dylib
2 ls: /usr/lib/libSystem.B.dylib: No such file or directory
```

Apple APIs, most notably `dlopen`, understand this and do the right thing if you supply the path of a library that moved into the cache. That's true for some, but not all, command-line tools, for example:

```
1 % dyld_info -exports /usr/lib/libSystem.B.dylib
2 /usr/lib/libSystem.B.dylib [arm64e]:
3   -exports:
4     offset      symbol
5     ...
6     0x5B827FE8  _mach_init_routine
7 % nm /usr/lib/libSystem.B.dylib
8 .../nm: error: /usr/lib/libSystem.B.dylib: No such file or directory
```

When the linker creates a Mach-O image, it adds a bunch of helpful information to that image, including:

- The target platform
- The **deployment target**, that is, the minimum supported version of that platform
- Information about the tools used to build the image, most notably, the SDK version
- A build UUID

For more information about the build UUID, see TN3178 [Checking for and resolving build UUID problems](#). To dump the other information, run `vttool`.

In some cases the OS uses the SDK version of the main executable to determine whether to enable new behaviour or retain old behaviour for compatibility purposes. You might see this referred to as *compiled against SDK X*. I typically refer to this as a **linked-on-or-later check**.

Apple tools support the concept of **autolinking**. When your code uses a symbol from a module, the compiler inserts a reference (using the `LC_LINKER_OPTION` load command) to that module into the resulting object file (`.o`). When you link with that object file, the linker adds the referenced module to the list of modules that it searches when resolving symbols.

Autolinking is obviously helpful but it can also cause problems, especially with cross-platform code. For information on how to enable and disable it, see the [Build settings reference](#).

Developer Forums

Mach-O uses a **two-level namespace** to resolve symbols. This improves both performance and reliability but it precludes certain techniques that might work on other platforms. For example, you can't define a function called `printf` and expect it to 'see' calls from other dynamic libraries because those libraries import the version of `printf` from `libSystem`.

To help folks who rely on techniques like this, macOS supports a **flat namespace** compatibility mode. This has numerous sharp edges — for an example, see the posts on [this thread](#) — and it's best to avoid it where you can. If you're enabling the flat namespace as part of a developer tool, search the 'net for *dyld interpose* to learn about an alternative technique.

Search for a topic, subtopic, or tag

Post



Sep 2022

WARNING Dynamic linker interposing is not documented as API. While it's a useful technique for developer tools, do not use it in products you ship to end users.

Apple platforms use **DWARF**. When you compile a file, the compiler puts the debug info into the resulting object file. When you link a set of object files into an executable, dynamic library, or bundle for distribution, the linker does not include this debug info. Rather, debug info is stored in a separate **debug symbols** document package. This has the extension `.dSYM` and is created using `dsymutil`. Use `symbols` to learn about the symbols in a file. Use `dwarfdump` to get detailed information about DWARF debug info. Use `atos` to map an address to its corresponding symbol name.

Different languages use different [name mangling](#) schemes:

- C, and all later languages, add a leading underscore (`_`) to distinguish their symbols from assembly language symbols.
- C++ uses a complex name mangling scheme. Use the `c++filt` tool to undo this mangling.
- Likewise, for Swift. Use `swift demangle` to undo this mangling.

For a bunch more info about symbols in Mach-O, see [Understanding Mach-O Symbols](#). This includes a discussion of **weak references** and **weak definition**. If your code is referencing a symbol unexpectedly, see [Determining Why a Symbol is Referenced](#).

To remove symbols from a Mach-O file, run `strip`. To hide symbols, run `nmedit`.

It's common for linkers to divide an object file into sections. You might find data in the data section and code in the text section (**text** is an old Unix term for code). Mach-O uses **segments** and **sections**. For example, there is a text segment (`__TEXT`) and within that various sections for code (`__TEXT > __text`), constant C strings (`__TEXT > __cstring`), and so on.

Over the years there have been some *really* good talks about linking and libraries at WWDC, including:

- WWDC 2023 Session 10268 [Meet mergeable libraries](#)
- WWDC 2022 Session 110362 [Link fast: Improve build and launch times](#)
- WWDC 2022 Session 110370 [Debug Swift debugging with LLDB](#)
- WWDC 2021 Session 10211 [Symbolication: Beyond the basics](#)
- WWDC 2019 Session 416 [Binary Frameworks in Swift](#) — Despite the name, this covers XCFrameworks in depth.
- WWDC 2018 Session 415 [Behind the Scenes of the Xcode Build Process](#)
- WWDC 2017 Session 413 [App Startup Time: Past, Present, and Future](#)
- WWDC 2016 Session 406 [Optimizing App Startup Time](#)

Note The older talks are no longer available from Apple, but you may be able to find transcripts out there on the 'net.

Historically Apple published a document, *Mac OS X ABI Mach-O File Format Reference*, or some variant thereof, that acted as the definitive reference to the Mach-O file format. This document is no longer available from Apple. If you're doing serious work with Mach-O, I recommend that you find an old copy. It's definitely out of date, but there's no better place to get a high-level introduction to the concepts. The [Mach-O Wikipedia page](#) has a link to an archived version of the document.

For the most up-to-date information about Mach-O, see the declarations and doc comments in `<mach-o/loader.h>`.

Revision History

- **2025-08-04** Added a link to [Determining Why a Symbol is Referenced](#).
- **2025-06-29** Added information about autolinking.
- **2025-05-21** Added a note about the legacy Mach-O stub library format (`MH_DYLIB_STUB`).
- **2025-04-30** Added a specific reference to the man pages for the TBD format.
- **2025-03-01** Added a link to [Understanding Mach-O Symbols](#). Added a link to TN3178 [Checking for and resolving build UUID problems](#). Added a summary of

Sep 2022



the information available via `vtool`. Discussed linked-on-or-later checks. Explained how Mach-O uses segments and sections. Explained the old (`-classic`) and new (`-llvm-`) tool variants. Referenced the Mach-O man page. Added basic info about the `strip` and `nmedit` tools.

- **2025-02-17** Expanded the discussion of dynamic library identification.
- **2024-10-07** Added some basic information about the dynamic linker shared cache.
- **2024-07-26** Clarified the description of the expected load address for Mach-O images.
- **2024-07-23** Added a discussion of position-independent images and the image slide.
- **2024-05-08** Added links to the demangling tools.
- **2024-04-30** Clarified the requirement to use the standard dynamic linker.
- **2024-03-02** Updated the discussion of static frameworks to account for Xcode 15 changes. Removed the link to WWDC 2018 Session 415 because it no longer works)-:
- **2024-03-01** Added the WWDC 2023 session to the list of sessions to make it easier to find. Added a reference to [Using a Link Map to Track Down a Symbol's Origin](#). Made other minor editorial changes.
- **2023-09-20** Added a link to [Dynamic Library Identification](#). Updated the names for the static linker implementations (`-ld_prime` is no more!). Removed the *beta* epithet from Xcode 15.
- **2023-06-13** Defined the term *Mach-O image*. Added sections for both the static and dynamic linkers. Described the two big new features in Xcode 15: mergeable libraries and dependency verification.
- **2023-06-01** Add a reference to `tapi-analyze`.
- **2023-05-29** Added a discussion of the two-level namespace.
- **2023-04-27** Added a mention of the `size` tool.
- **2023-01-23** Explained the compile-time and run-time roles of a framework. Made other minor editorial changes.
- **2022-11-17** Added an explanation of TAPI.
- **2022-10-12** Added links to Mach-O documentation.
- **2022-09-29** Added info about `.dSYM` files. Added a few more links to WWDC sessions.
- **2022-09-21** First posted.

↑ Boost



This site contains user submitted content, comments and opinions and is for informational purposes only. Apple disclaims any and all liability for the acts, omissions and conduct of any third parties in connection with or related to your use of the site. All postings and use of the content on this site are subject to the [Apple Developer Forums Participation Agreement](#) and Apple provided code is subject to the [Apple Sample Code License](#).



Platforms	Topics & Technologies	Resources	Programs
iOS	Accessibility	Documentation	Apple Developer Program
iPadOS	Accessories	Tutorials	Apple Developer Enterprise Program
macOS	App Extensions	Downloads	App Store Small Business Program
tvOS	App Store	Forums	MFi Program
watchOS	Audio & Video	Videos	News Partner Program
visionOS	Augmented Reality		Video Partner Program
	Design	Support	Security Bounty Program
Tools	Distribution	Support Articles	Security Research Device Program
Swift	Education	Contact Us	
SwiftUI	Fonts	Bug Reporting	Events
SF Symbols	Games	System Status	Meet with Apple
Swift Playground	Health & Fitness		Apple Developer Centers
TestFlight	In-App Purchase	Account	App Store Awards
Xcode	Localization	Apple Developer	Apple Design Awards
Xcode Cloud	Maps & Location	App Store Connect	Apple Developer Academies
	Machine Learning	Certificates, IDs, & Profiles	WWDC
	Open Source	Feedback Assistant	
	Security		
	Safari & Web		
To view the latest developer news, visit News and Updates .			
Copyright © 2026 Apple Inc. All rights reserved. Terms of Use Privacy Policy License Agreements			